



Laporan Praktikum Tugas 1 RESTful API Murni dengan Express.js

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Muhammad Luthfi Hilmy

NPM : 2428240032

Kelas : SI5B

**PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027**

Identitas Praktikum

Nama Mahasiswa	Muhammad Luthfi Hilmy
NPM	2428240032
Kelas	SI5B
Pertemuan Ke-	3
Topik / Judul Praktikum	Laporan Praktikum Tugas 1 RESTful API Murni dengan Express.js
Tanggal Pelaksanaan	2026-09-29
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Membuat project Express.js sebagai fondasi awal pembuatan aplikasi *backend*
2. Membuat routing RESTful untuk memetakan URL dan metode HTTP ke fungsi penanganan (*handler*)
3. Menerima request dari client melalui parameter rute (*req.params*), *query string* (*req.query*), dan *body request* (*req.body*)
4. Mengirimkan response murni dalam format JSON beserta penggunaan kode status HTTP yang sesuai
5. Mengimplementasikan RESTful API secara utuh dengan menggabungkan konsep *routing*, penanganan *request*, dan *response* JSON murni
6. Mengembangkan RESTful API murni untuk pengelolaan *resource* stok darah PMI (/blood-stocks) yang mencakup operasi CRUD lengkap, validasi data, *catch-all middleware* 404, serta *deployment* ke Vercel

II. Dasar Teori

A. Environment & Runtime (Node.js, NVM, dan NPM)

Node.js merupakan *runtime environment* JavaScript berbasis mesin V8 milik Google Chrome yang memungkinkan kode JavaScript dijalankan di luar peramban (*sisi server*). Node.js menjadi fondasi utama untuk menjalankan Express.js dan ekosistem paket npm. NVM (Node Version Manager) merupakan alat untuk menginstal dan mengelola beberapa versi Node.js secara fleksibel pada satu sistem, memastikan seluruh pengembang dalam tim bekerja pada versi Node.js yang konsisten. NPM (Node Package Manager) merupakan manajer paket bawaan Node.js yang digunakan untuk menginstal, memperbarui, dan mengelola dependensi/paket pustaka yang tercatat dalam berkas *package.json* dan *package-lock.json*

B. Express.js Framework

Express.js merupakan *framework* web minimalis dan fleksibel untuk Node.js yang mempermudah pembangunan *server* web dan RESTful API tanpa menulis banyak kode *boilerplate*. Fitur utamanya meliputi pemetaan rute (*routing*) sederhana untuk menghubungkan URL dan metode HTTP ke fungsi penanganan (*handler*), dukungan komponen perantara (*middleware*), serta fleksibilitas struktur proyek

C. Arsitektur RESTful API

REST (*Representational State Transfer*) adalah gaya arsitektur untuk merancang *web service* berbasis *resource* (sumber daya) yang diakses melalui protokol HTTP. Prinsip utamanya meliputi:

- a. Resource-Based: Setiap entitas data (seperti `/blood-stocks`) direpresentasikan melalui URL unik berupa kata benda jamak
- b. Stateless: Setiap permintaan (*request*) bersifat independen, sehingga *server* tidak menyimpan status sesi (*session state*) klien
- c. Pemetaan HTTP Method ke Operasi CRUD:
 - GET: Membaca seluruh koleksi data atau data spesifik berdasarkan ID tanpa mengubah data
 - POST: Membuat/menambahkan data baru ke *server*
 - PUT: Memperbarui/mengganti seluruh *field* data yang ada secara penuh berdasarkan ID
 - DELETE: Menghapus data spesifik berdasarkan ID

D. Anatomi Request, Response, dan Middleware Express.js

- Objek Request (`req`) digunakan untuk membaca data masuk dari klien melalui `req.params` (parameter rute dinamis), `req.query` (*query string*), dan `req.body` (data pada *body request*).
- Middleware `express.json()` Komponen perantara wajib yang mendaftarkan penguraian (*parsing*) JSON sebelum rute agar aplikasi dapat membaca `req.body`.
- Objek Response (`res`) dan Kode Status HTTP digunakan untuk mengirimkan respons kembali ke klien beserta status HTTP yang sesuai. 2xx (Berhasil): 200 OK (berhasil mengambil/mengubah data), 201 Created (berhasil membuat data baru), 204 No Content (berhasil menghapus data). 4xx / 5xx (Gagal/Error): 400 Bad Request, 404 Not Found (data/rute tidak ditemukan), atau 500 Internal Server Error

E. Deployment Server (Vercel)

Platform Vercel menyediakan lingkungan *cloud serverless* untuk mengeksekusi aplikasi Express.js. Aplikasi diekspor menggunakan `module.exports = app` dan dikonfigurasi melalui berkas `vercel.json` agar *serverless function* dapat memetakan rute secara otomatis

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	<i>Runtime environment</i> JavaScript untuk menjalankan <i>server</i> aplikasi sisi <i>backend</i>
2	NPM (Node Package Manager)	Manajer paket bawaan Node.js untuk mengelola instalasi dependensi proyek
3	Express.js	Version ^5.2.1 (<i>framework web</i> utama untuk membangun RESTful API)
4	Nodemon	Version ^3.1.14 (<i>devDependency</i> untuk memuat ulang <i>server</i> secara otomatis saat terjadi perubahan kode)
5	Visual Studio Code	<i>Code editor</i> utama yang digunakan untuk penulisan dan pengelolaan kode proyek
6	Git & GitHub	<i>Version control system</i> dan repositori publik (tugas1-restful-2428240032) untuk pengelolaan riwayat <i>commit</i>
7	Vercel	Platform <i>cloud deployment</i> berbasis <i>serverless</i> untuk meng-host RESTful API secara <i>online</i>
8	Postman	Alat penguji API (<i>API testing tool</i>) untuk menjalankan 10 skenario pengujian <i>endpoint</i>

IV. Langkah Kerja

1. Inisialisasi Proyek dan Instalasi Dependensi

Langkah awal diawali dengan membuat folder proyek baru bernama tugas1-restful-2428240032 melalui terminal dan masuk ke dalam direktori tersebut. Proyek Node.js diinisialisasi dengan mengeksekusi perintah `npm init -y` untuk menghasilkan berkas konfigurasi `package.json`. Selanjutnya, *framework* Express.js diinstal sebagai dependensi utama melalui perintah `npm install express`, dan paket Nodemon diinstal sebagai dependensi pengembangan (*devDependency*) via `npm install --save-dev nodemon` agar *server* otomatis memuat ulang saat terjadi perubahan kode.

2. Konfigurasi `package.json`, `.gitignore`, dan `vercel.json`

Berkas `package.json` diperbarui pada bagian `scripts` dengan menambahkan skrip `"start": "node app.js"` untuk lingkungan produksi dan `"dev": "nodemon app.js"` untuk lingkungan pengembangan. Berkas `.gitignore` dibuat untuk memastikan folder `node_modules/` dan berkas sensitif `.env` tidak ikut terunggah ke repositori GitHub. Selain itu, berkas `vercel.json` dibuat pada direktori utama (*root*) dengan konfigurasi `@vercel/node` untuk mengatur alur *deployment serverless* ke platform Vercel.

3. Inisialisasi Server, Middleware, dan Data Awal (`app.js`)

Berkas *entry point* utama `app.js` dibuat. Pada baris awal, pustaka Express diimpor dan *middleware* `app.use(express.json())` didaftarkan wajib agar *server* Express dapat mengurai (*parsing*) data masukan berformat JSON dari `req.body`. Selanjutnya, variabel penyimpanan data sementara berbasis memori (*in-memory array*) bernama `bloodStocks` dibuat berisi 18 record data awal stok darah PMI lengkap dengan variabel penghitung ID otomatis `let nextId = 19`;

4. Implementasi Endpoint Informasi API (`GET /`)

Membuat rute akar `GET /` menggunakan `app.get('/', ...)` yang mengembalikan respons murni JSON berisi informasi identitas mahasiswa (Nama: Muhammad Luthfi Hilmy, NPM: 2428240032, Topik: 7 - Stok Darah PMI) serta daftar seluruh *endpoint* API yang tersedia. *Endpoint* ini berfungsi untuk memastikan *server* berjalan dengan baik sekaligus menampilkan dokumentasi singkat saat diakses via peramban.

5. Pembuatan Rute CRUD dan Filter Query String (`/blood-stocks`)

Membangun enam rute utama sesuai standar konvensi RESTful API:

1. `GET /blood-stocks`: Mengambil seluruh koleksi 18 data stok darah dalam bentuk *array* JSON murni.
2. `GET /blood-stocks/:id`: Mengambil detail satu data berdasarkan `req.params.id`. Jika ID ditemukan, mengembalikan objek data dengan status 200 OK; jika tidak ada, mengembalikan status 404 Not Found.

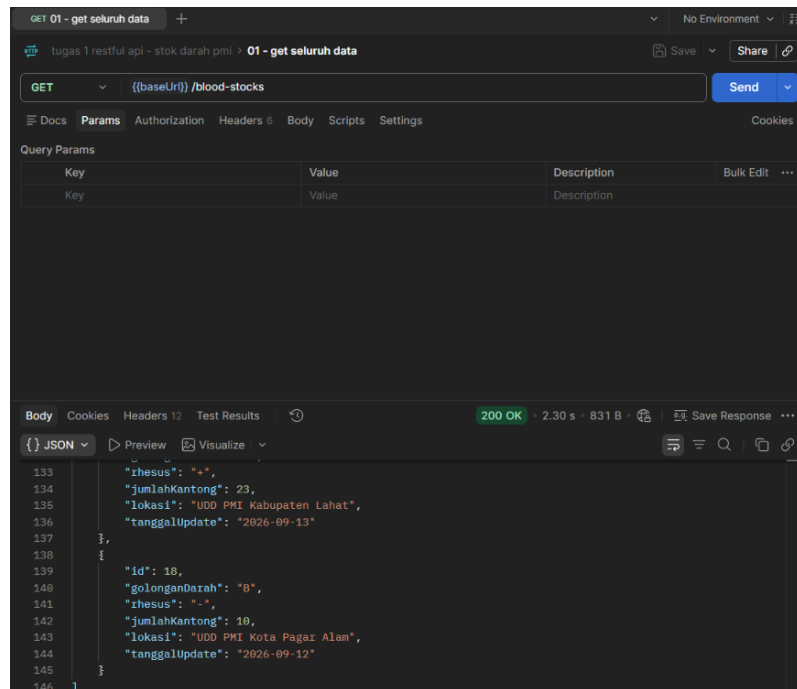
3. GET `/blood-stocks?golonganDarah=O`: Menggunakan `req.query` untuk menyaring data berdasarkan parameter golongan darah dan mengembalikan hasil berupa *array* data yang cocok.
 4. POST `/blood-stocks`: Menerima data baru dari `req.body`, memberikan id otomatis dari `nextId++`, memasukkannya ke *array*, dan mengembalikan respons berstruktur `{ status: "success", message: "...", data: ... }` dengan status 201 Created.
 5. PUT `/blood-stocks/:id`: Melakukan pembaruan/penggantian penuh seluruh *field* data berdasarkan ID dan mengembalikan data hasil pembaruan dengan status 200 OK.
 6. DELETE `/blood-stocks/:id`: Menghapus elemen data dari *array* menggunakan `.splice()` berdasarkan ID dan mengembalikan respons status 200 OK dengan data: null.
6. Penerapan Fungsi Validasi Data Masukan
- Membuat fungsi validasi (misalnya `validateBloodStock`) yang diterapkan pada rute POST dan PUT. Fungsi ini memeriksa keberadaan seluruh *field* wajib (`golonganDarah`, `rhesus`, `jumlahKantong`, `lokasi`, `tanggalUpdate`), memastikan nilai `golonganDarah` hanya berupa "A", "B", "AB", atau "O", nilai `rhesus` berupa "+" atau "-", `jumlahKantong` bertipe angka, dan `tanggalUpdate` berformat YYYY-MM-DD. Jika validasi gagal, *server* menolak permintaan dengan kode status 400 Bad Request.
7. Penanganan Middleware Catch-All 404
- Menambahkan *middleware catch-all* di bagian paling bawah pendaftaran rute pada `app.js` untuk menangkap seluruh permintaan rute yang tidak terdaftar (misalnya GET `/endpoint-salah`). *Middleware* ini mengembalikan respons JSON `{ "status": "error", "message": "Endpoint tidak ditemukan", "data": null }` dengan kode status 404 Not Found.
8. Pengaturan Ekspor Serverless dan Listener Port Lokal
- Agar aplikasi dapat berjalan baik di komputer lokal maupun di platform Vercel, kode penjalana `server` `app.listen()` dibungkus dalam kondisi `if (process.env.NODE_ENV !== 'production')`. Selanjutnya, *instance* aplikasi Express diekspor di baris akhir berkas menggunakan `module.exports = app`;
9. Version Control Git dan Deployment Vercel
- Proyek diinisialisasi dengan Git (`git init`), lalu dilakukan penyimpanan riwayat perubahan bertahap (`commit`) minimal 5 kali. Repositori kemudian di-push ke GitHub publik dengan nama tugas1-restful-2428240032. Langkah diakhiri dengan mengimpor repositori GitHub tersebut ke platform Vercel hingga aplikasi berhasil ter-deploy secara online.

V. Hasil dan Pembahasan

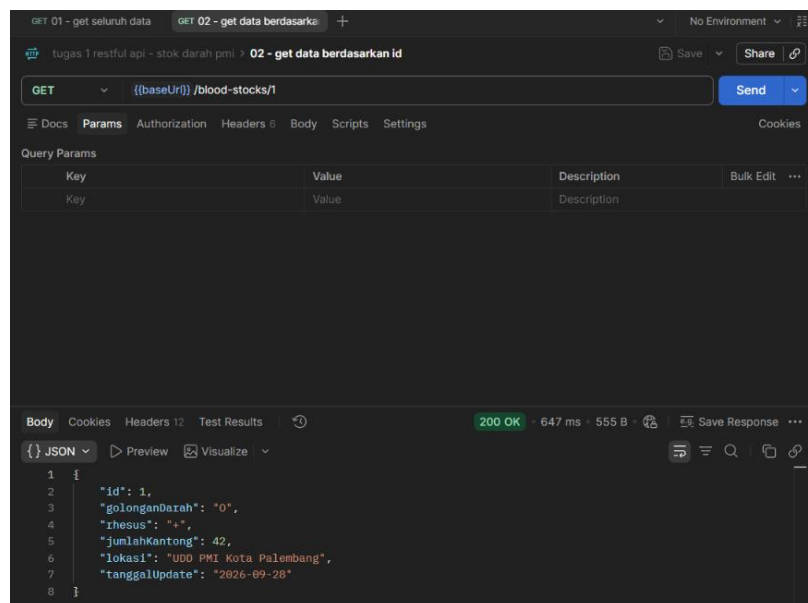
1. Tabel Hasil Pengujian API (Vercel & Postman Collection)

Berikut adalah rangkuman hasil pengujian 10 skenario *endpoint* API yang dijalankan pada URL *deployment* Vercel (<https://tugas1-restful-2428240032.vercel.app>) menggunakan koleksi Postman 2428240032_Postman.json:

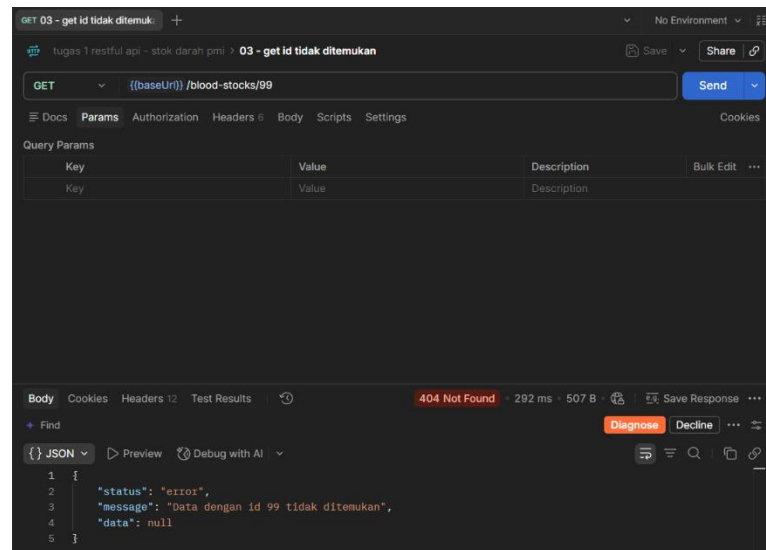
No.	Method	Endpoint / URL	Data Request	Status Diharapkan	Status Hasil
1.	GET	/blood-stocks	(Tanpa Body)	200 OK	200 OK
2.	GET	/blood-stocks/1	(Tanpa Body)	200 OK	200 OK
3.	GET	/blood-stocks/99	(Tanpa Body)	404 Not Found	404 Not Found
4.	GET	/blood-stocks?golonganDarah=O	(Tanpa Body)	200 OK	OK
5.	POST	/blood-stocks	Body JSON Lengkap	201 Created	201 Created
6.	POST	/blood-stocks	Field Wajib Kosong	400 Bad Request	400 Bad Request
7.	PUT	/blood-stocks/1	Body JSON Lengkap	200 OK	200 OK
8.	PUT	/blood-stocks/99	Body JSON Lengkap	404 Not Found	404 Not Found
9.	DELETE	/blood-stocks/1	(Tanpa Body)	200 OK	200 OK
10.	DELETE	/blood-stocks/99	(Tanpa Body)	404 Not Found	404 Not Found



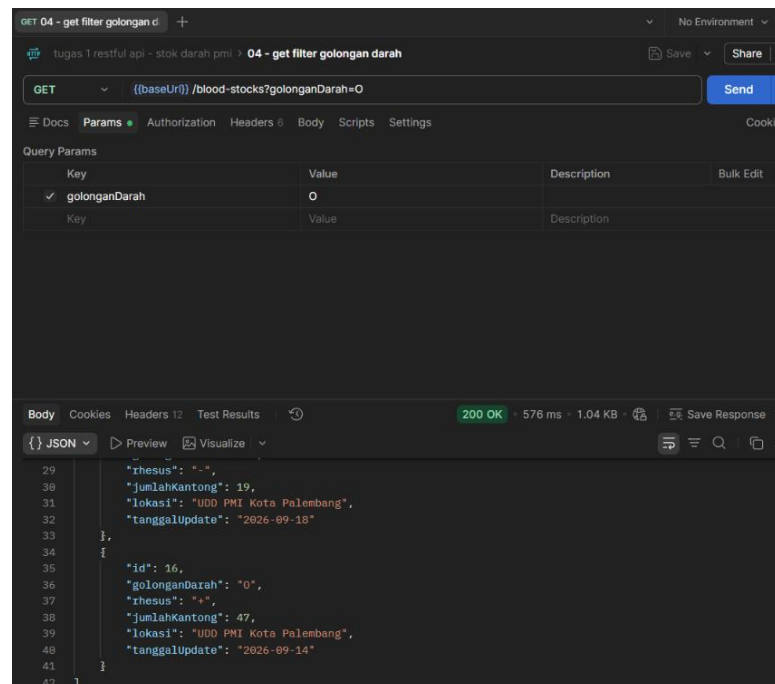
Gambar 5.1 Hasil Pengujian GET /blood-stocks (Mengambil 18 Data Awal)



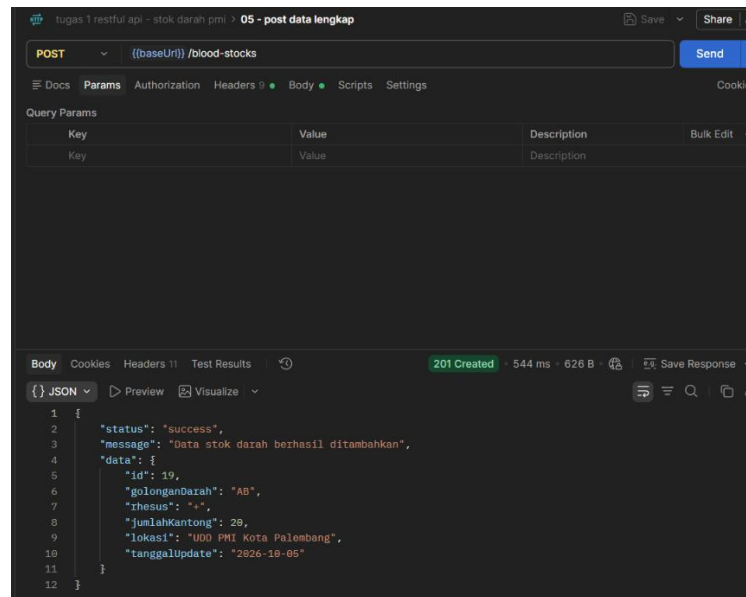
Gambar 5.2 Hasil Pengujian GET /blood-stocks/1 (Detail Data ID 1 - Status 200 OK)



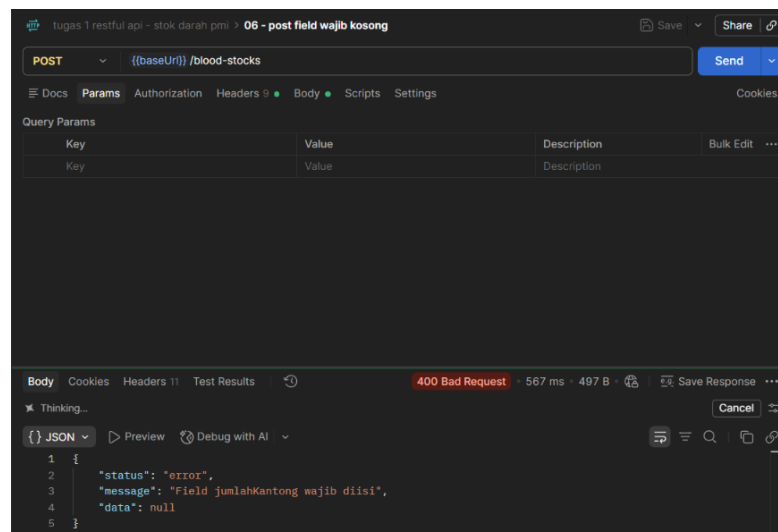
Gambar 5.3 Hasil Pengujian GET /blood-stocks/99 (ID Tidak Ditemukan - Status 404 Not Found)



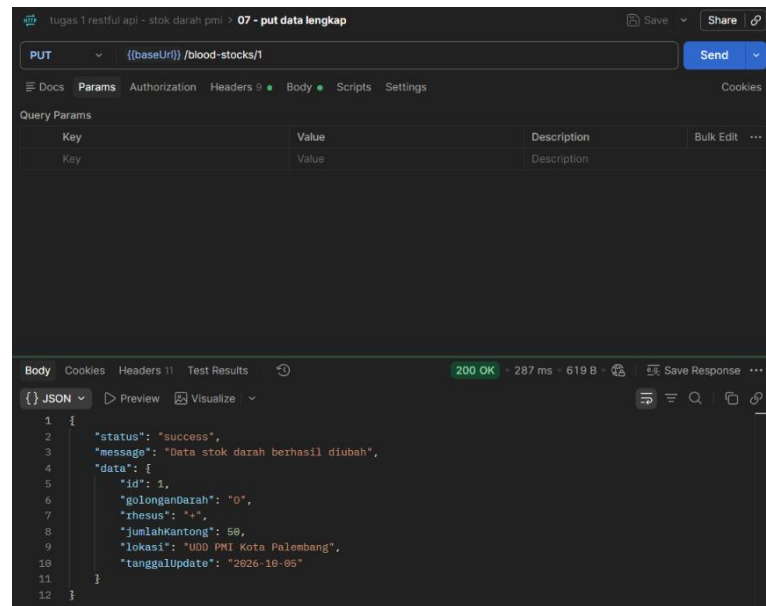
Gambar 5.4 Hasil Pengujian GET /blood-stocks?golonganDarah=0 (Filter Query String)



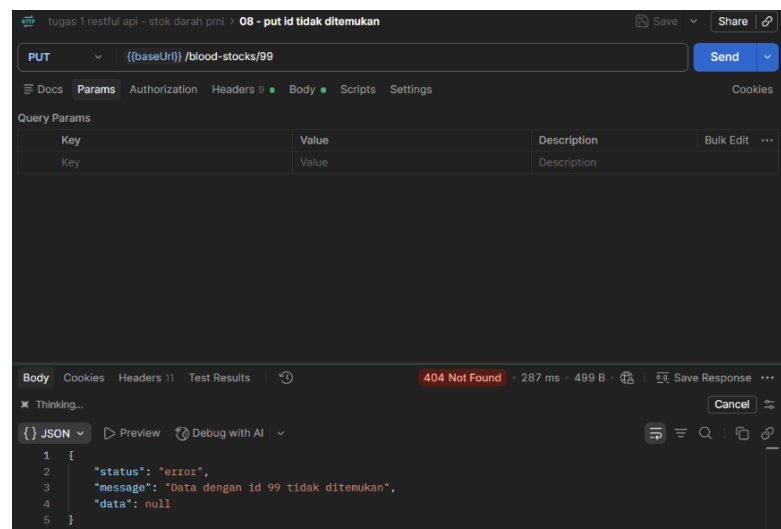
Gambar 5.5 Hasil Pengujian POST /blood-stocks (Tambah Data Baru ID 19 - Status 201 Created)



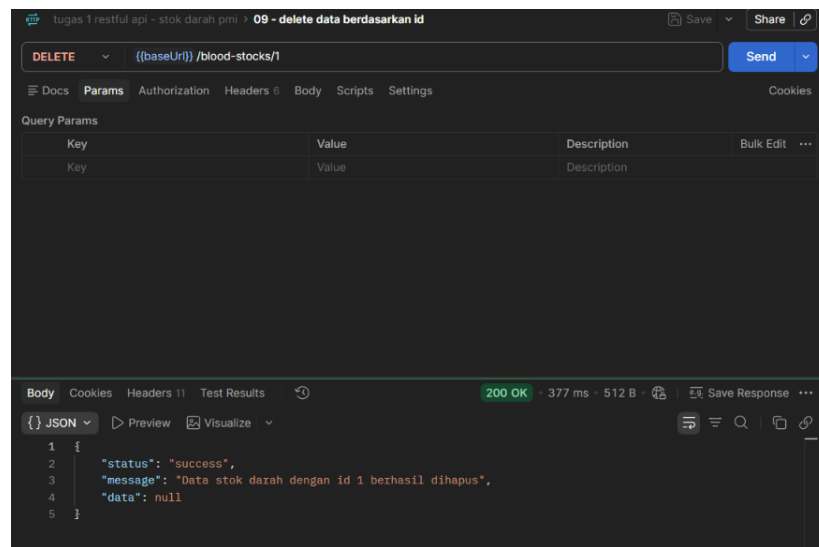
Gambar 5.6 Hasil Pengujian POST Validasi Gagal (Field Wajib Kosong - Status 400 Bad Request)



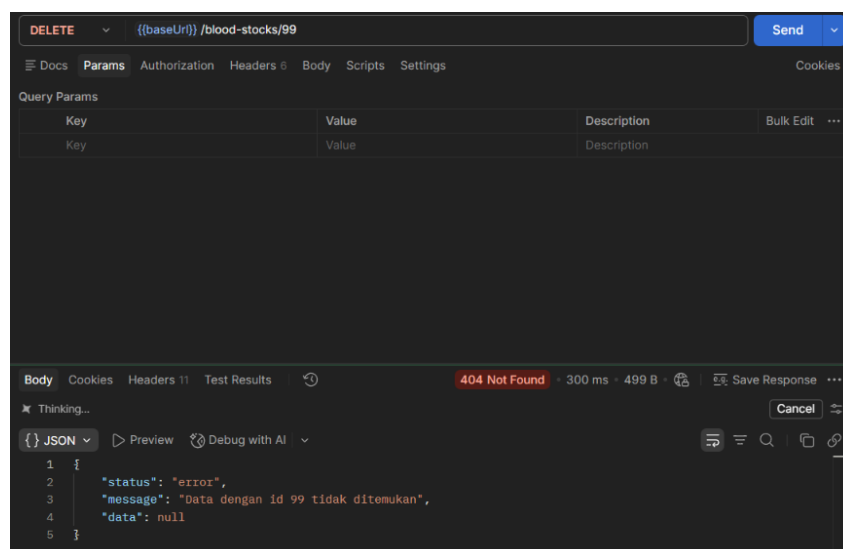
Gambar 5.7 Hasil Pengujian *PUT /blood-stocks/1* (Penggantian Data Penuh - Status 200 OK)



Gambar 5.8 Hasil Pengujian *PUT /blood-stocks/99* (Ubah Data Gagal - Status 404 Not Found)



Gambar 5.9 Hasil Pengujian DELETE /blood-stocks/1 (Hapus Data Sukses - Status 200 OK)



Gambar 5.10 Hasil Pengujian DELETE /blood-stocks/99 (Hapus Data Gagal - Status 404 Not Found)

2. Pembahasan Analisis Implementasi API

a. Pengujian Endpoint Read (GET All, GET By ID, dan Query Filter)

Skenario GET /blood-stocks mengambil seluruh koleksi data stok darah PMI. Sesuai konvensi tugas, *endpoint* ini mengembalikan data murni dalam bentuk *array* JSON berisi 18 record tanpa dibungkus objek status maupun message.

Skenario GET /blood-stocks/:id memanfaatkan parameter rute dinamis req.params.id. Ketika ID 1 dipanggil, *server* mencari data via fungsi `.find()` dan mengembalikan satu objek JSON ber-status 200 OK. Jika dipanggil dengan ID 99 yang tidak tersimpan, *server* secara

tepat mengembalikan kode status 404 Not Found dengan pembungkus JSON { "status": "error", "message": "Data dengan id 99 tidak ditemukan", "data": null }.

Skenario GET /blood-stocks?golonganDarah=O menggunakan *query string* req.query.golonganDarah. *Server* menyaring *array* menggunakan .filter() dan mengembalikan *array* berisi 5 data yang cocok. Apabila nilai filter tidak ditemukan, *server* tetap mengembalikan *array* kosong [] dengan status 200 OK.

b. Pengujian Endpoint Create (POST) dan Validasi Input Data

Skenario POST /blood-stocks: Diberikan *body request* JSON lengkap (golonganDarah, rhesus, jumlahKantong, lokasi, tanggalUpdate). *Server* mengurai data melalui *middleware* express.json(), menetapkan ID otomatis (nextId = 19), dan memasukkannya ke *array*. Respons dibungkus menggunakan struktur standar { "status": "success", "message": "...", "data": { ... } } ber-status 201 Created.

Skenario Validasi Status 400 Bad Request: Jika salah satu *field* wajib tidak dikirim atau jenis datanya tidak sesuai (misal: golonganDarah diisi selain "A"|"B"|"AB"|"O" atau rhesus selain "+"|"-"), fungsi validateBloodStock menolak permintaan dan membalas dengan kode status 400 Bad Request beserta pesan kesalahan yang informatif.

c. Pengujian Endpoint Update (PUT) dan Penggantian Data Penuh

Sesuai ketentuan RESTful API, metode PUT melakukan penggantian seluruh *field* data (*full replacement*). Saat mengirimkan permintaan ke /blood-stocks/1 dengan seluruh *field* baru, *server* terlebih dahulu mencari indeks data melalui .findIndex(). Jika ID ditemukan dan seluruh *field* valid, data lama diganti penuh dan merespons dengan status 200 OK beserta data hasil pembaruan. Apabila ID tidak ditemukan (seperti pada ID 99), *server* mengembalikan respon 404 Not Found.

d. Pengujian Endpoint Delete (DELETE) dan Response JSON Murni

Permintaan DELETE /blood-stocks/1 mencari data berdasarkan ID dan menghapusnya dari *array* menggunakan .splice(index, 1). Berbeda dengan latihan dasar yang mengembalikan status 204 No Content tanpa *body*, pada Tugas 1 ini operasi DELETE yang berhasil mengembalikan kode status 200 OK beserta respons JSON bermuatan pesan konfirmasi dan data: null. Jika ID tidak ada, dikembalikan respons error 404 Not Found.

e. Analisis Middleware Catch-All 404 dan Pengaruh Serverless Vercel

Middleware Catch-All 404 mencegah Express mengirimkan balasan HTML bawaan saat URL yang salah diakses (misal: GET /endpoint-salah). *Middleware* ini menjamin seluruh respons dari aplikasi selalu konsisten berformat JSON murni. Karakteristik serverless vercel karena disimpan pada *in-memory array* tanpa basis data eksternal, data baru yang ditambahkan via POST dapat kembali ke 18 data awal jika wadah *serverless* Vercel berada dalam kondisi *idle*. Hal ini merupakan sifat normal platform *cloud serverless* dan telah sesuai dengan batasan penugasan.

VI. Kendala dan Solusi

No.	Kendala Teknis	Solusi dan Penanganan
1	Pembacaan req.body bernilai undefined saat pengujian metode POST dan PUT.	Mendaftarkan <i>middleware bawaan</i> <code>app.use(express.json())</code> pada baris awal berkas <code>app.js</code> sebelum pendefinisian rute agar Express dapat mengurai data JSON masukan.
2	Data baru hasil POST/PUT kembali ke 18 data awal secara otomatis setelah beberapa saat di Vercel.	Hal ini merupakan sifat alami lingkungan <i>cloud serverless</i> Vercel (<i>in-memory array</i> tanpa basis data). Kondisi ini normal, sesuai dengan batasan penugasan tanpa basis data eksternal, dan tidak mengurangi nilai.
3	Permintaan URL yang salah awalnya mengembalikan respons teks/HTML bawaan dari Express.js.	Menambahkan <i>middleware catch-all 404</i> di akhir berkas <code>app.js</code> untuk memastikan semua rute tak dikenal membalas dengan JSON murni { "status": "error", "message": "Endpoint tidak ditemukan", "data": null }.
4	Terjadi kegagalan validasi jenis data masukan (seperti huruf golongan darah kecil atau format tanggal salah).	Membuat fungsi <code>validateBloodStock</code> untuk memvalidasi <i>field</i> wajib, mengecek pilihan enum (A B AB O dan + -), memastikan jumlahKantong bertipe <i>number</i> , dan mengecek format tanggal YYYY-MM-DD.

VII. Kesimpulan

Berdasarkan pelaksanaan praktikum dan penyelesaian Tugas 1 (RESTful API Stok Darah PMI), dapat disimpulkan bahwa:

1. Implementasi RESTful API Murni: Aplikasi *backend* untuk pengelolaan stok darah PMI (/blood-stocks) berhasil dibangun berbasis Node.js dan Express.js secara murni tanpa menggunakan basis data eksternal maupun *template engine* HTML.
2. Kepatuhan Konvensi dan Operasi CRUD: Seluruh 6 *endpoint* utama (pembacaan seluruh data, detail ID, penambahan data, pembaruan penuh, penghapusan data, dan filter *query string* golonganDarah) berjalan dengan benar sesuai standar konvensi RESTful API.
3. Konsistensi Respons JSON & Kode Status: Seluruh respons API dikirimkan dalam format JSON murni. Operasi GET mengembalikan data langsung, sedangkan operasi POST, PUT, DELETE, dan penanganan *error* menggunakan pembungkus standar { status, message, data } beserta kode status HTTP yang tepat (200 OK, 201 Created, 400 Bad Request, dan 404 Not Found).

4. Validasi Data & Error Handling: Fungsi validasi masukan berhasil menyaring data yang tidak lengkap atau tidak valid, serta *middleware catch-all* 404 menjamin tidak ada respons HTML yang terkirim ke klien.
5. Deployment & Version Control: Proyek berhasil dikelola menggunakan Git/GitHub (tugas1-restful-2428240032) dengan riwayat *commit* bertahap, sukses di-deploy secara *online* ke platform Vercel, dan lolos pengujian 10 skenario Postman (2428240032_Postman.json).

Lampiran

Tautan Repository GitHub:

<https://github.com/fiyoraa/tugas1-restful-2428240032>

Tautan Deployment:

<https://tugas1-restful-2428240032.vercel.app/>
